



دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش کار پروژه

بررسی و بهینه‌سازی پایگاه داده PostgreSQL

طراحی پایگاه داده‌ها

نیمسال دوم ۰۴-۰۵

استاد: دکتر مریم رمضانی

نویسندگان:

یاسمن کاویانپور - 403102514

امیرمهدی وزیری - 402106756

سبحان آرام - 403110614

۱. گزارش تحقیق

۱.۱. پارامترهای پیکربندی PostgreSQL

فایل پیکربندی `postgresql.conf` هسته اصلی مدیریت منابع در PostgreSQL است. تنظیم دقیق این پارامترها تاثیر مستقیمی بر نحوه مدیریت حافظه، عملیات ورودی/خروجی (I/O) و تصمیمات برنامه‌ریز کوثری (Query Planner) دارد. در ادامه به تفصیل به بررسی این پارامترها می‌پردازیم [1, 2]:

- **shared_buffers**: این پارامتر تعیین‌کننده میزان حافظه اشتراکی است که PostgreSQL برای کش کردن (Cache) صفحات داده استفاده می‌کند. از آنجا که سیستم‌عامل نیز داده‌ها را در حافظه نهان خود نگه می‌دارد، اختصاص تمام حافظه رم به این پارامتر منطقی نیست و منجر به پدیده بافرینگ دوگانه (Double Buffering) می‌شود. مقدار پیشنهادی و بهینه برای این پارامتر معمولاً بین ۲۵٪ تا ۴۰٪ از کل حافظه RAM سرور است. افزایش این مقدار باعث می‌شود صفحات بیشتری در حافظه باقی بمانند و نیاز به خواندن مجدد از دیسک (I/O) به شدت کاهش یابد [2].

- **effective_cache_size**: برخلاف `shared_buffers`، این پارامتر هیچ حافظه‌ای را به صورت واقعی تخصیص نمی‌دهد. بلکه صرفاً یک متغیر آماری و تخمینی برای برنامه‌ریز کوثری است تا بداند مجموعاً چه مقدار حافظه (حافظه خود دیتابیس به علاوه کش سیستم‌عامل) برای نگهداری داده‌ها در دسترس است. اگر این مقدار بزرگ تنظیم شود، برنامه‌ریز متوجه می‌شود که احتمال یافتن داده‌ها در رم بسیار بالاست، بنابراین استفاده از اسکن ایندکس (Index Scan) را ترجیح می‌دهد. مقادیر کوچک باعث می‌شوند برنامه‌ریز تصور کند که خواندن تصادفی از دیسک هزینه بالایی دارد و به سمت اسکن ترتیبی (Sequential Scan) سوق پیدا کند [2].

- **work_mem**: این پارامتر حداکثر حافظه مجاز برای هر عملیات مرتب‌سازی (Sort) یا درهم‌سازی (Hash Join) را پیش از سرریز شدن داده‌ها به فایل‌های موقت روی دیسک (Spill to Disk) مشخص می‌کند. دقت کنید که این مقدار برای هر عملیات در هر کوثری تخصیص می‌یابد. بنابراین یک کوثری پیچیده با چندین عملیات Sort ممکن است چندین برابر این مقدار را اشغال کند. مقادیر بالای این پارامتر سرعت کوثری‌های تحلیلی را به شدت افزایش می‌دهد، اما در سیستم‌هایی با تراکشن‌های همزمان زیاد، تنظیم بیش از حد آن می‌تواند منجر به پر شدن کامل حافظه (OOM) شود [2].

- **maintenance_work_mem**: این حافظه مختص عملیات‌های نگهداری سیستم نظیر `VACUUM`، `ALTER`، `INDEX CREATE` و `TABLE REINDEX` است. از آنجا که این فرآیندها عموماً توسط ادمین و در زمان‌های خاصی اجرا می‌شوند و تعداد آن‌ها به صورت همزمان بسیار کم است، می‌توان مقدار این پارامتر را بسیار بیشتر از `work_mem` در نظر گرفت. مقدار بالا در این پارامتر باعث می‌شود ساخت ایندکس‌های حجیم تماماً در حافظه رم انجام شود و سرعت فرآیند چندین برابر گردد [2].

- **wal_buffers**: در معماری پایگاه داده، تغییرات پیش از اعمال روی فایل‌های اصلی، در فایل‌های Write-Ahead Log (WAL) نوشته می‌شوند تا در صورت قطعی برق، داده‌ها از دست نروند. `wal_buffers` حافظه‌ای است که این لاگ‌ها پیش از همگام‌سازی (Flush) با دیسک در آن قرار می‌گیرند. در سناریوهای بارگذاری انبوه داده (Bulk Load) یا تراکشن‌های همزمان بسیار زیاد، افزایش این بافر (مثلاً تا ۱۶ مگابایت) باعث می‌شود تا رکوردهای بیشتری تجمع شده و تعداد دفعات نوشتن روی دیسک کاهش یابد [1].

- **min_wal_size** و **max_wal_size**: این پارامترها محدوده‌ای را برای حجم فایل‌های WAL پیش از اجرای اجباری یک چک‌پوینت (Checkpoint) تعیین می‌کنند. هنگامی که حجم لاگ‌ها به `max_wal_size` می‌رسد، دیتابیس مجبور می‌شود تمام صفحات کثیف (Dirty Pages) را روی دیسک بنویسد. مقادیر کوچک باعث چک‌پوینت‌های پی‌درپی و فشار شدید I/O می‌شوند. از سوی دیگر، مقادیر بزرگتر (مثلاً چند گیگابایت) عملکرد درج‌های انبوه را به شدت بهبود می‌بخشند، اما در صورت بروز خرابی در سیستم (Crash)، زمان بازیابی و Recovery طولانی‌تر خواهد بود [1].

- **seq_page_cost** و **random_page_cost**: این دو پارامتر ثابت‌های ریاضی برای برنامه‌ریز کوثری هستند تا هزینه خواندن یک صفحه از دیسک را تخمین بزنند. `seq_page_cost` (خواندن متوالی) به صورت پیش‌فرض ۰.۱ و `random_page_cost` (خواندن تصادفی) ۰.۴ است. این نسبت ۴ به ۱ مربوط به دوران دیسک‌های سخت مکانیکی (HDD) است که حرکت هد دیسک برای خواندن تصادفی زمان‌بر بود. امروزه با استفاده از درایوهای حالت جامد (SSD)، این تفاوت بسیار ناچیز است. در نتیجه، کاهش `random_page_cost` به اعدادی نظیر ۱.۱ باعث می‌شود برنامه‌ریز هزینه‌های اسکن ایندکس را واقعی‌تر ارزیابی کرده و از اسکن‌های ترتیبی غیرضروری پرهیز کند [1].

- **enable_seqscan**، **enable_bitmapscan** و **enable_indexscan**: این متغیرها به عنوان کلیدهای قطع و وصل برای روش‌های مختلف اسکن عمل می‌کنند. در عمل، خاموش کردن آن‌ها روش اسکن را به طور کامل غیرممکن نمی‌کند، بلکه هزینه

تخمینی آن را برای برنامه ریز به شکل مصنوعی و نجومی بالا می برد تا از آن دوری کند. کاربرد اصلی آن ها در محیط های عملیاتی نیست، بلکه ابزاری قدرتمند برای دیباگ کردن و تست عملکرد برنامه ریز است تا دریابیم اگر یک ایندکس خاص در دسترس نبود، پایگاه داده چه مسیر اجرایی جایگزینی را انتخاب می کرد [1].

۲.۱. ایندکس ها در PostgreSQL

پایگاه داده PostgreSQL ساختارهای متنوعی را برای ایندکس گذاری فراهم کرده است که هر یک از الگوریتم های متفاوتی بهره می برند [1, 3]:

- **B-Tree Index**: ایندکس پیش فرض سیستم است که بر پایه ساختار درختی متوازن (Balanced Tree) کار می کند و داده ها را به صورت مرتب شده نگه می دارد. **ساختار داده**: درخت جستجوی چندسطحی که در آن تمامی گره های برگ در یک عمق قرار دارند. **موارد استفاده**: بهترین انتخاب برای عملگرهای برابری (=) و جستجوهای بازه ای (BETWEEN, <, >, ≤, ≥). **مزایا**: تطبیق پذیری بالا، پشتیبانی کامل از مرتب سازی داده ها (ORDER BY) و کارایی عالی برای انواع داده های استاندارد (اعداد، رشته ها). **معایب**: برای جستجو درون ساختارهای پیچیده و مرکب (نظیر آرایه ها، متون طولانی و اشیاء JSON) کاربردی ندارد [1].
- **Hash Index**: ایندکسی است که صرفاً برای عملیات برابری دقیق بهینه شده است. **ساختار داده**: جدول هش (Hash Table) که مقادیر را به کدهای هش ۳۲ بیتی نگاشت می کند. **موارد استفاده**: تنها برای عملگر تساوی دقیق (=). **مزایا**: در برخی موارد جستجوی تساوی، ممکن است کمی سریع تر و کوچکتر از B-Tree عمل کند. **معایب**: عدم پشتیبانی از عملگرهای بازه ای، عدم پشتیبانی از مرتب سازی، و محدودیت در ایجاد ایندکس های چندستونی [1].
- **GiST Index (Generalized Search Tree)**: به جای یک ساختار داده واحد، GiST یک زیرساخت و چارچوب تعمیم یافته است که به توسعه دهندگان اجازه می دهد قوانین ایندکس گذاری دلخواه خود را تعریف کنند. **ساختار داده**: درختی متوازن که هر گره می تواند نشان دهنده ویژگی های پیچیده تری مانند مستطیل های محیطی (Bounding Boxes) باشد. **موارد استفاده**: داده های هندسی و مکانی (GIS)، جستجوی بازه های زمانی دارای هم پوشانی و جستجوی نزدیک ترین همسایه (KNN). **مزایا**: انعطاف پذیری بی نظیر برای تعریف عملگرهای سفارشی. **معایب**: فرآیند ساخت و به روز رسانی آن پیچیده و کندتر است و از آنجا که ساختار آن در برخی موارد همراه با اتلاف (Lossy) است، نتایج جستجو گاهی نیازمند اعتبارسنجی مجدد در سطر اصلی هستند [1].
- **GIN Index (Generalized Inverted Index)**: ایندکس معکوس که به طور خاص برای مدیریت مقادیری که حاوی چندین عنصر هستند طراحی شده است. **ساختار داده**: به جای نگاشت یک سطر به یک مقدار، هر عنصر داخلی (مثلاً کلمات یک متن یا کلیدهای یک شیء) را به لیستی از سطری که آن عنصر را در بر دارند (Posting List) نگاشت می کند. **موارد استفاده**: جستجوی فول تکست، آرایه ها و ساختارهای JSONB. **مزایا**: سرعت خارق العاده در جستجوی عضویت و دربرگیری (Containment). **معایب**: هزینه بسیار بالای به روز رسانی؛ زیرا درج یک رکورد جدید می تواند به معنای تغییر در ده ها گره مختلف در ایندکس باشد [3].

نتیجه گیری برای JSONB: ساختار **GIN Index** سازگارترین و قدرتمندترین ایندکس برای استفاده بر روی فیلدهای JSONB است، زیرا ماهیت تو در تو و تو و کلید-مقدار JSONB، نیازمند جستجوی سریع روی عناصر داخلی است که GIN دقیقاً برای همین هدف توسعه یافته است [3].

۳.۱. ساختار JSONB در PostgreSQL

PostgreSQL دو نوع داده برای ذخیره سازی اسناد JSON ارائه می دهد، اما تفاوت های معماری آن ها بنیادین است [1, 4]:

- **تفاوت بین JSON و JSONB**: نوع داده JSON یک کپی دقیق از متن ورودی را (همراه با فاصله ها و ترتیب کلیدها) ذخیره می کند. در نتیجه، عملیات نوشتن روی آن بسیار سریع است اما در هر بار جستجو، پایگاه داده باید متن را از ابتدا تجزیه (Parse) کند. در مقابل، JSONB داده ها را در یک قالب باینری، تجزیه شده و بهینه سازی شده ذخیره می کند. فواصل اضافی حذف شده و ترتیب کلیدها ممکن است تغییر کند. اگرچه زمان درج JSONB به دلیل فرآیند تبدیل کمی بیشتر است، اما پردازش، خواندن و جستجو روی آن به مراتب سریع تر انجام می شود [1].
- **نحوه ذخیره سازی و مکانیزم TOAST**: اسناد JSONB به صورت رکوردهای باینری در جداول ذخیره می شوند. چنانچه حجم یک شیء JSONB از آستانه مشخصی (معمولاً ۲ کیلوبایت) فراتر رود، مکانیزم **TOAST (The Oversized-Attribute Storage Technique)** به صورت خودکار فعال می شود. این مکانیزم داده ها را فشرده کرده و آن ها را به قطعات کوچکتر شکسته و در یک جدول کمکی مخفی ذخیره می کند، و تنها یک اشاره گر (Pointer) در سطر اصلی باقی می گذارد تا از افت عملکرد در اسکن های متوالی جلوگیری شود [4].

● عملگرها و توابع مهم JSONB:

- > : برای استخراج یک فیلد یا عنصر آرایه که خروجی آن مجدداً از نوع JSONB است.
- >> : مشابه عملگر قبلی، با این تفاوت که خروجی را مستقیماً به عنوان متن (Text) برمی‌گرداند.
- @> : عملگر دربرگیری (Containment). این عملگر بررسی می‌کند که آیا شیء سمت چپ، تمام مسیرها و مقادیر شیء سمت راست را در بر دارد یا خیر.
- ? : بررسی می‌کند که آیا یک کلید متنی خاص در بالاترین سطح شیء JSONB وجود دارد.
- jsonb_set() : تابعی برای به‌روزرسانی یا جایگزینی مقدار یک کلید خاص درون شیء بدون نیاز به بارگذاری مجدد کل ساختار در سمت برنامه.
- jsonb_agg() : تابع تجمعی که چندین سطر پایگاه داده را در قالب یک آرایه JSONB خروجی می‌دهد [1].

- نحوه ایجاد ایندکس روی فیلدهای JSONB: برای جستجوی کارآمد، از ایندکس GIN استفاده می‌شود. دو کلاس عملگر (Operator Class) برای این کار وجود دارد. روش پیش‌فرض (jsonb_ops) هم کلیدها و هم مقادیر را ایندکس می‌کند و از عملگرهای متعددی پشتیبانی می‌کند. اما اگر هدف صرفاً استفاده از عملگر دربرگیری (@>) باشد، استفاده از کلاس jsonb_path_ops ایندکسی به مراتب کوچکتر و سریع‌تر تولید می‌کند. دستور ایجاد آن به شکل زیر است [3]:

```
CREATE INDEX idx_name ON table USING GIN (column_name jsonb_path_ops);
```

- چه زمانی باید از JSONB به جای جدول نرمال استفاده کرد؟: استفاده از JSONB زمانی توجیه مهندسی دارد که با داده‌های بدون شمای ثابت (Schema-less) روبرو باشیم. به عنوان مثال، اگر فیلدهای متادیتا (Metadata) برای رکوردهای مختلف کاملاً متفاوت باشند، استفاده از جداول نرمال منجر به ایجاد ده‌ها ستون اکثراً NULL (Sparse Data) و جداول رابطه‌ای متعدد می‌شود. اما اگر ساختار داده کاملاً مشخص است و کوئری‌ها دائماً نیاز به پیوند (Join)، فیلتر کردن و گروه‌بندی بر اساس فیلدهای داخلی دارند، استفاده از جداول نرمال شده همراه با کلیدهای خارجی و ایندکس‌های B-Tree عملکرد بهتری هم در تضمین جامعیت داده (Data Integrity) و هم در سرعت خواهد داشت [1].

۴.۱. استفاده از PostgreSQL به عنوان صف پیام

- پیاده‌سازی یک صف پیام (Message Queue) مستقیماً درون دیتابیس رابطه‌ای، مزایا و محدودیت‌های تئوریک خاص خود را دارد [1, 5]:
- مکانیزم **LISTEN / NOTIFY**: این قابلیت داخلی PostgreSQL امکان پیاده‌سازی مدل انتشار/اشتراک (Pub/Sub) را فراهم می‌کند. یک فرآیند می‌تواند با دستور channel_name LISTEN به یک کانال گوش دهد و فرآیند دیگر با استفاده از NOTIFY channel_name, 'payload' پیامی را ارسال کند. این فرآیند بسیار سبک و سریع است و به محض پایان یافتن تراکنش فرستنده (Commit)، گیرنده‌ها پیام را به صورت ناهمگام (Asynchronous) دریافت می‌کنند [1].
- محدودیت‌های **LISTEN / NOTIFY** به عنوان صف: این مکانیزم برای یک صف مطمئن و پایدار کافی نیست زیرا کاملاً بدون حافظه (Stateless و Volatile) است. اگر در لحظه ارسال NOTIFY، مصرف‌کننده‌ای موقتاً قطع شده باشد، پیام برای همیشه از دست می‌رود. علاوه بر این، NOTIFY پیام را به تمامی نشست‌های متصل ارسال می‌کند (Broadcast) که این امر با الگوی مصرف‌کنندگان رقیب (Competing Consumers) که در آن هر پیام باید دقیقاً توسط یک کارگر پردازش شود، در تضاد است [1].
- راه‌اندازی صف با استفاده از جداول و **SKIP LOCKED**: راهکار اصولی برای پیاده‌سازی صف در PostgreSQL، ایجاد یک جدول مشخص (با فیلدهایی نظیر شناسه پیام، بدنه پیام، وضعیت، زمان ایجاد و دفعات تلاش) است. برای مصرف کردن پیام‌ها، کوئری باید بتواند رکوردهای با وضعیت pending را پیدا کرده و همزمان آن‌ها را برای پردازش قفل کند. در اینجا دستور SELECT ... LOCKED SKIP UPDATE FOR ... به عنوان قفل تپنده صف وارد عمل می‌شود [5].
- عبارت **FOR UPDATE SKIP LOCKED** و جلوگیری از **Race Condition**: به طور معمول، UPDATE FOR ... سطر را قفل می‌کند تا هیچ تراکنش دیگری نتواند آن را تغییر دهد. اگر چندین کارگر همزمان یک سطر را درخواست کنند، همگی پشت قفل بلوکه می‌شوند. عبارت کلیدی LOCKED SKIP UPDATE در دستور می‌دهد که هر سطر که توسط تراکنش دیگری قفل شده است را فوراً نادیده بگیرد و به سراغ سطر آزاد بعدی برود. این معماری به صورت کامل مشکل Race Condition و قفل‌شدگی را حل می‌کند، زیرا کارگرهای متعدد می‌توانند به صورت کاملاً موازی و بدون هیچ‌گونه اتلاف وقتی، دسته‌های متمایزی از پیام‌ها را از صف برداشته و پردازش کنند [5].

- مزایا و معایب در مقایسه با راه‌حل‌های تخصصی (مانند Kafka یا RabbitMQ): مزایا: مهم‌ترین مزیت، کاهش پیچیدگی زیرساخت است. نیازی به نگهداری سرورها و سیستم‌های نرم‌افزاری مجزا نیست. همچنین، این معماری ثبات تراکنشی (ACID) خارق‌العاده‌ای ارائه می‌دهد؛ شما می‌توانید در یک تراکنش واحد، یک پیام را پردازش کرده، رکوردی را در جدول دیگری به‌روزرسانی کنید و پیام را از صف حذف (یا وضعیت آن را تغییر) دهید. اگر هر بخشی از این مسیر با خطا مواجه شود، تمام عملیات بازمی‌گردد (Rollback). معایب: در سیستم‌هایی با توان عملیاتی بسیار بالا (ده‌ها هزار پیام در ثانیه)، جداول دیتابیس به شدت تحت فشار قرار می‌گیرند. هر تغییر وضعیت پیام نیازمند نوشتن نسخه‌های جدید رکوردها است که پدیده Table Bloat را به دنبال داشته و سربار سنگینی برای مکانیزم جمع‌آوری زباله (VACUUM) و I/O دیسک ایجاد می‌کند. در این مقیاس‌های کلان، سیستم‌های تخصصی مقیاس‌پذیری و عملکرد بهتری از خود نشان می‌دهند [1].

۲. گزارش عملی

۱.۲. پیاده‌سازی توابع صف پیام

هسته اصلی معماری پردازش ناهمگام در این پروژه، مبتنی بر چهار تابع کلیدی است که به زبان PL/pgSQL در پایگاه داده پیاده‌سازی شده‌اند. این توابع وظیفه مدیریت چرخه حیات پیام‌ها (ماشین حالت بین وضعیت‌های pending، processing، done و failed) و تضمین هم‌روندی ایمن (Concurrency Control) را بر عهده دارند. در ادامه کد دقیق و تحلیل هر یک از این توابع آورده شده است.

۱. تابع Enqueue (درج در صف)

این تابع نقطه ورود داده‌ها به صف است. تابع enqueue نام صف (p_queue_name) و بار داده (p_payload) که از نوع JSONB است را دریافت کرده و پس از اعتبارسنجی اولیه، در جدول imdb.message_queue درج می‌کند. پیام‌ها در بدو ورود با وضعیت pending و تعداد تلاش اولیه (attempt_count) صفر ذخیره می‌شوند. همچنین حداکثر دفعات مجاز برای پردازش یک پیام (max_attempts) به صورت پیش‌فرض برابر ۵ تنظیم شده است تا از حلقه‌های تکرار بی‌نهایت (در صورت وجود داده‌های فاسد) جلوگیری شود.

```
CREATE OR REPLACE FUNCTION imdb.enqueue(
    p_queue_name TEXT,
    p_payload JSONB
) RETURNS BIGINT AS $$
DECLARE
    v_message_id BIGINT;
BEGIN
    IF p_queue_name IS NULL OR BTRIM(p_queue_name) = '' THEN
        RAISE EXCEPTION 'queue name must not be null or empty'
        USING ERRCODE = '22023';
    END IF;

    IF p_payload IS NULL THEN
        RAISE EXCEPTION 'payload must not be null'
        USING ERRCODE = '22004';
    END IF;

    INSERT INTO imdb.message_queue (
        queue_name,
        payload,
        status,
        attempt_count,
        max_attempts,
        available_at,
        created_at,
        updated_at
```

```

)
VALUES (
    p_queue_name,
    p_payload,
    'pending',
    0,
    5,
    NOW(),
    NOW(),
    NOW()
)
RETURNING id INTO v_message_id;

RETURN v_message_id;
END;
$$ LANGUAGE plpgsql;

```

۲. تابع Dequeue (برداشت ایمن از صف)

این تابع پیچیده‌ترین و حیاتی‌ترین بخش صف پیام است که مقیاس‌پذیری و مصرف موازی (بدون Race Condition) را تضمین می‌کند. تابع dequeue پس از بررسی پارامترهای ورودی (مانند اندازه دسته و زمان مشاهده‌پذیری)، در دو مرحله (با استفاده از CTE‌های زبان SQL) عمل می‌کند:

۱. **پاک‌سازی پیام‌های منقضی‌شده:** ابتدا پیام‌هایی که در وضعیت processing قرار دارند اما زمان مشاهده‌پذیری (visibility_timeout) آن‌ها به پایان رسیده و تعداد تلاش‌هایشان از حد مجاز (max_attempts) گذشته است را پیدا کرده و وضعیت آن‌ها را برای همیشه به failed تغییر می‌دهد تا منابع سیستم آزاد شوند.

۲. **برداشت پیام‌های جدید:** سپس با استفاده از عبارت کلیدی **FOR UPDATE SKIP LOCKED**، دسته‌ای از پیام‌های pending (یا پیام‌های processing که مهلتشان تمام شده اما هنوز شانس تلاش مجدد دارند) را قفل می‌کند. این قفل‌گذاری باعث می‌شود سایر مصرف‌کننده‌ها (Consumers) بدون انتظار برای آزادسازی قفل، به سراغ رکوردهای بعدی بروند. در نهایت وضعیت این پیام‌ها به processing تغییر یافته و شمارنده تلاش آن‌ها یک واحد افزایش می‌یابد.

```

CREATE OR REPLACE FUNCTION imdb.dequeue(
    p_queue_name TEXT,
    p_batch_size INTEGER DEFAULT 10,
    p_visibility_timeout INTERVAL DEFAULT INTERVAL '30 seconds'
) RETURNS TABLE(msg_id BIGINT, payload JSONB) AS $$
BEGIN
    IF p_queue_name IS NULL OR BTRIM(p_queue_name) = '' THEN
        RAISE EXCEPTION 'queue name must not be null or empty'
        USING ERRCODE = '22023';
    END IF;

    IF p_batch_size IS NULL OR p_batch_size < 1 OR p_batch_size > 10000 THEN
        RAISE EXCEPTION 'batch size must be between 1 and 10000'
        USING ERRCODE = '22023';
    END IF;

    IF p_visibility_timeout IS NULL OR p_visibility_timeout <= INTERVAL '0 seconds' THEN
        RAISE EXCEPTION 'visibility timeout must be greater than zero'
        USING ERRCODE = '22023';
    END IF;

```

```

-- Finalize abandoned messages that have already exhausted all attempts.
WITH exhausted AS (
    SELECT mq.id
    FROM imdb.message_queue AS mq
    WHERE mq.queue_name = p_queue_name
        AND mq.status = 'processing'
        AND mq.processing_started_at <= NOW() - p_visibility_timeout
        AND mq.attempt_count >= mq.max_attempts
    ORDER BY mq.id
    FOR UPDATE SKIP LOCKED
)
UPDATE imdb.message_queue AS mq
SET status = 'failed',
    failed_at = NOW(),
    processing_started_at = NULL,
    last_error = COALESCE(
        NULLIF(mq.last_error, ''),
        'visibility timeout expired after maximum attempts'
    ),
    updated_at = NOW()
FROM exhausted
WHERE mq.id = exhausted.id;

RETURN QUERY
WITH selected AS (
    SELECT mq.id
    FROM imdb.message_queue AS mq
    WHERE mq.queue_name = p_queue_name
        AND mq.attempt_count < mq.max_attempts
        AND (
            (mq.status = 'pending' AND mq.available_at <= NOW())
            OR
            (
                mq.status = 'processing'
                AND mq.processing_started_at <= NOW() - p_visibility_timeout
            )
        )
    ORDER BY mq.id
    LIMIT p_batch_size
    FOR UPDATE SKIP LOCKED
)
UPDATE imdb.message_queue AS mq
SET status = 'processing',
    attempt_count = mq.attempt_count + 1,
    processing_started_at = NOW(),
    completed_at = NULL,
    failed_at = NULL,
    last_error = CASE
        WHEN mq.status = 'processing'
        THEN COALESCE(NULLIF(mq.last_error, ''), 'visibility timeout expired;

```

```

        message reclaimed')
    ELSE mq.last_error
END,
    updated_at = NOW()
FROM selected
WHERE mq.id = selected.id
RETURNING mq.id, mq.payload;
END;
$$ LANGUAGE plpgsql;

```

۳. تابع Ack (تایید پردازش موفق)

تابع ack برای تایید موفقیت‌آمیز بودن پردازش یک پیام فراخوانی می‌شود. این تابع وضعیت پیام را از processing به done تغییر می‌دهد. یکی از ویژگی‌های مهم این پیاده‌سازی، بی‌اثری (Idempotency) آن است؛ به این معنا که اگر پیامی پیش‌تر تایید شده باشد یا وضعیت آن تغییر کرده باشد، این تابع به جای پرتاب خطای سیستمی (که باعث از کار افتادن برنامه مصرف‌کننده می‌شود)، به سادگی و به صورت امن از آن عبور می‌کند.

```

CREATE OR REPLACE FUNCTION imdb.ack(
    p_msg_id BIGINT
) RETURNS VOID AS $$
BEGIN
    IF p_msg_id IS NULL THEN
        RAISE EXCEPTION 'message id must not be null'
        USING ERRCODE = '22004';
    END IF;

    UPDATE imdb.message_queue
    SET status = 'done',
        completed_at = COALESCE(completed_at, NOW()),
        processing_started_at = NULL,
        failed_at = NULL,
        updated_at = NOW()
    WHERE id = p_msg_id
    AND status IN ('processing', 'done');
END;
$$ LANGUAGE plpgsql;

```

۴. تابع Nack (ثبت خطا و تلاش مجدد)

در صورتی که مصرف‌کننده در پردازش پیام (مثلاً به دلیل تداخل تراکنش‌ها یا خطای اعتبارسنجی داده) با خطا مواجه شود، تابع nack فراخوانی می‌گردد. این تابع دارای یک منطق تصمیم‌گیری هوشمند برای مدیریت خطاهاست:

- **خطاهای دائمی (Permanent):** اگر دلیل خطا با عبارت [permanent] آغاز شود یا تعداد تلاش‌ها به max_attempts رسیده باشد، پیام مستقیماً به وضعیت failed منتقل شده و زمان شکست آن (failed_at) ثبت می‌گردد.
- **عقب‌نشینی نمایی (Exponential Backoff):** در صورت بروز خطاهای موقت (غیر دائمی)، وضعیت پیام به pending برمی‌گردد. اما برای جلوگیری از فشار مضاعف و رگباری به دیتابیس، زمان در دسترس قرار گرفتن مجدد آن (available_at) با استفاده از فرمول توان دوم (2, power(mq.attempt_count, 2)) - 1، (0) به صورت نمایی به تاخیر می‌افتد و این تاخیر به سقف ۳۰۰ ثانیه (۵ دقیقه) محدود شده است.

```

CREATE OR REPLACE FUNCTION imdb.nack(
    p_msg_id BIGINT,

```

```

        p_reason TEXT DEFAULT ''
    ) RETURNS VOID AS $$
BEGIN
    IF p_msg_id IS NULL THEN
        RAISE EXCEPTION 'message id must not be null'
            USING ERRCODE = '22004';
    END IF;

    UPDATE imdb.message_queue AS mq
    SET status = CASE
        WHEN mq.status IN ('done', 'failed') THEN mq.status
        WHEN COALESCE(p_reason, '') LIKE '[permanent]%'
            OR mq.attempt_count >= mq.max_attempts
            THEN 'failed'
        ELSE 'pending'
    END,
    available_at = CASE
        WHEN mq.status IN ('done', 'failed') THEN mq.available_at
        WHEN COALESCE(p_reason, '') LIKE '[permanent]%'
            OR mq.attempt_count >= mq.max_attempts
            THEN mq.available_at
        ELSE NOW() + make_interval(
            secs => LEAST(
                300,
                CAST(power(2, GREATEST(mq.attempt_count - 1, 0)) AS INTEGER)
            )
        )
    END,
    processing_started_at = CASE
        WHEN mq.status IN ('done', 'failed') THEN mq.processing_started_at
        ELSE NULL
    END,
    failed_at = CASE
        WHEN mq.status = 'failed' THEN COALESCE(mq.failed_at, NOW())
        WHEN mq.status = 'done' THEN mq.failed_at
        WHEN COALESCE(p_reason, '') LIKE '[permanent]%'
            OR mq.attempt_count >= mq.max_attempts
            THEN NOW()
        ELSE NULL
    END,
    last_error = CASE
        WHEN mq.status IN ('done', 'failed') THEN mq.last_error
        ELSE LEFT(COALESCE(p_reason, ''), 4000)
    END,
    updated_at = CASE
        WHEN mq.status IN ('done', 'failed') THEN mq.updated_at
        ELSE NOW()
    END
    WHERE mq.id = p_msg_id;
END;
$$ LANGUAGE plpgsql;

```

۲.۲. برنامه‌های تولیدکننده (Producer) و مصرف‌کننده (Consumer)

معماری خط لوله داده (Data Pipeline) در این پروژه به دو اسکریپت مجزای پایتون تقسیم شده است که وظیفه انتقال اطلاعات از فایل‌های خام TSV به جداول نهایی پایگاه داده را بر عهده دارند. در پیاده‌سازی این دو برنامه، مفاهیم پیشرفته‌ای نظیر تراکنش‌های اتمیک، درج انبوه (Bulk Insert)، مکانیزم فشار پس (Backpressure) و مدیریت خطای هوشمند پیاده‌سازی شده است. از آنجا که کدهای کامل در فایل‌های تحویلی موجود هستند، در اینجا صرفاً به تحلیل دقیق منطق و ارجاع به خطوط و توابع کلیدی آن‌ها می‌پردازیم.

اسکریپت تولیدکننده (Producer.py)

وظیفه اصلی این اسکریپت خواندن فایل‌های فشرده و تزریق سریع آن‌ها به صورت پیام‌های JSONB به داخل صف است. این اسکریپت با در نظر گرفتن محدودیت‌های حافظه و دیسک طراحی شده است:

- **خواندن جریانی (Streaming Read):** در تابع `open_dataset`، فایل‌های حجیم دارای پسوند `.gz` با استفاده از کتابخانه `gzip` به صورت جریانی باز می‌شوند. سپس ماژول `csv.DictReader` بدون استخراج کامل فایل در حافظه رم یا دیسک، سطر به سطر داده‌ها را خوانده و پردازش می‌کند.
- **بسته‌بندی و درج انبوه (Batching):** برای جلوگیری از سربار شبکه‌ای ناشی از فراخوانی تکی توابع پایگاه داده، سطرها درون یک لیست موقت (`batch`) ذخیره می‌شوند. زمانی که حجم این لیست به `batch_size` (پیش‌فرض ۵۰۰) رسید، تابع `send_batch` فراخوانی می‌شود. در این تابع، با استفاده از ابزار قدرتمند `psycopg2.extras.execute_values`، تمام پیام‌های آن دسته طی یک کوئری واحد و در یک رفت‌و برگشت شبکه‌ای (Round-trip) به تابع `imdb.enqueue` ارسال می‌گردند.
- **مکانیزم فشار پس (Backpressure):** از آنجا که سرعت خواندن از دیسک بسیار بیشتر از سرعت پردازش و درج در جداول رابطه‌ای است، صف ممکن است به سرعت متورم شده و دیسک را پر کند. برای حل این مشکل، تابع `wait_for_queue_capacity` پیاده‌سازی شده است. این تابع به صورت مداوم تعداد پیام‌های در حال انتظار (In-flight) را می‌شمارد. اگر این تعداد از سقف تعیین‌شده (`high_water`، پیش‌فرض ۲۰,۰۰۰) عبور کند، اسکریپت Producer متوقف شده و منتظر می‌ماند تا Consumerها پیام‌ها را مصرف کنند و حجم صف به زیر خط استاندارد (`low_water`، پیش‌فرض ۱۰,۰۰۰) برسد.
- **تاب‌آوری و ذخیره پیشرفت (Resilience):** اگر فرآیند بارگذاری در میانه راه قطع شود، خواندن مجدد ده‌ها میلیون سطر از ابتدا غیرمنطقی است. با استفاده از توابع `load_progress` و `save_progress`، اسکریپت به صورت دوره‌ای شماره آخرین سطر پردازش‌شده و متادیتاهای فایل (حجم و زمان تغییر) را در جدول `imdb.producer_progress` ثبت می‌کند. با این مکانیزم، در صورت کرش کردن برنامه، تولیدکننده دقیقاً از همان سطری که متوقف شده بود فرآیند را از سر می‌گیرد.

اسکریپت مصرف‌کننده (Consumer.py)

این اسکریپت وظیفه استخراج پیام‌ها از صف، تجزیه (Parse) مقادیر JSONB و نگاشت ایمن آن‌ها به جداول نهایی را بر عهده دارد.

- **حلقه اصلی و برداشت دسته‌ای:** در تابع `run_consumer`، اسکریپت به صورت پیوسته تابع `imdb.dequeue` را با اندازه دسته مشخص فراخوانی می‌کند. پیام‌های دریافت شده سپس به تابع `process_claimed_batch` سپرده می‌شوند.
- **گروه‌بندی هوشمند (Grouping):** از آنجا که پیام‌های درون صف ممکن است متعلق به جداول مختلفی (مثل `title_basics` و `title_ratings`) باشند، پیام‌ها بر اساس نام جدول مقصد (`table_name`) گروه‌بندی می‌شوند تا بتوان درج انبوه (Bulk Insert) را برای هر جدول به صورت کاملاً مجزا و بهینه انجام داد.
- **درج با انزوای خطا (Fallback Mechanism):** یکی از چالش‌های اصلی درج دسته‌ای (Batch Insert) این است که وجود حتی یک سطر نامعتبر باعث شکست کل دسته می‌شود. در تابع `insert_group_with_fallback`، ابتدا تلاش می‌شود کل دسته با استفاده از دستور `execute_values` درون یک `SAVEPOINT` درج شود. اگر دیتابیس خطایی (مانند نقض محدودیت) پرتاب کند، اسکریپت کل دسته را بازگردانی (Rollback) می‌کند. سپس به یک مسیر جایگزین (Fallback) وارد شده و داده‌ها را سطر به سطر در `SAVEPOINT`های مجزا درج می‌کند. این کار باعث می‌شود دقیقاً مشخص شود کدام سطر مشکل دارد. سطرهای سالم با موفقیت درج و `ACK` می‌شوند، و سطر فاسد به صورت دائم `NACK` شده و ایزوله می‌گردد.

• **بی‌اثری و تراکنش‌های اتمیک (Idempotency):** تمامی کوئری‌های درج در دیکشنری TABLE_INSERT_TEMPLATES ساختار ON CONFLICT DO NOTHING نوشته شده‌اند. این امر تضمین می‌کند که اگر پیامی دو بار پردازش شود، رکوردهای تکراری نادیده گرفته می‌شوند. همچنین کل چرخه برداشت از صف (Dequeue)، درج در جدول نهایی (Insert) و تایید پیام (ACK / NACK) به صورت یک تراکنش اتمیک انجام می‌شود؛ به طوری که تمامی این مراحل با دستور connection.commit() نهایی می‌شوند. اگر در هر کجای این چرخه خطایی رخ دهد، تراکنش بازگردانی شده و پیام به صورت امن در صف باقی می‌ماند.

• **مدیریت فضای دیسک با Purge:** برای جلوگیری از رشد بی‌رویه جدول صف پیام، تابع purge_completed_messages به صورت دوره‌ای فراخوانی می‌شود تا پیام‌هایی که وضعیت done دارند را به صورت دسته‌ای و با محدودیت مشخص (purge_limit) از جدول message_queue حذف (یا به اصطلاح جاروب) کند.

۳.۲. نتایج اجرای ۸ سناریوی پرس‌وجو

پس از پیاده‌سازی و بارگذاری داده‌ها توسط اسکریپت‌های پایتون، هشت سناریوی تحلیلی درخواست‌شده در صورت پروژه (موجود در فایل queries.sql) بر روی پایگاه داده اجرا شدند. به منظور سنجش دقیق تاثیر استراتژی ایندکس‌گذاری، این کوئری‌ها یک بار پیش از ساخت ایندکس‌ها (حالت Baseline) و یک بار پس از اعمال فایل indexes.sql (حالت Optimized) با دستور EXPLAIN (ANALYZE, BUFFERS) اجرا شدند.

جدول زیر مقایسه زمان اجرای واقعی (Execution Time) کوئری‌ها را در این دو حالت نشان می‌دهد:

شرح سناریو	زمان خط پایه (ms)	زمان بهینه شده (ms)	درصد تغییر
۱. میانگین امتیاز فیلم‌ها به تفکیک ژانر	0.156	0.140	۲٪ بهبود
۲. ۱۰ فیلم برتر با بیشترین تعداد رأی	0.032	0.028	۱۲٪ بهبود
۳. کارگردانان با بیشترین تعداد فیلم	0.089	0.097	۸٪ افت
۴. روند سالانه میانگین رتبه (از سال ۲۰۰۰)	0.040	0.037	۷٪ بهبود
۵. بازیگران چندژانره (بیش از ۵ ژانر)	0.035	0.038	۸٪ افت
۶. سریال‌های با بیشترین فصل	0.037	0.043	۱۶٪ افت
۷. محبوب‌ترین ترکیب دوتایی ژانرها	460.997	510.269	۱۰٪ افت
۸. طولانی‌ترین فیلم‌ها به تفکیک ژانر	852.968	695.562	۱۸٪ بهبود

۴.۲. تحلیل خروجی EXPLAIN ANALYZE

بررسی زمان‌های اجرا و مسیرهای انتخابی توسط برنامه‌ریز کوئری (Query Planner) حاوی نکات فنی بسیار مهمی است. در ادامه به تحلیل عمیق لاگ‌های اجرایی برای سه سناریوی کلیدی می‌پردازیم:

تحلیل جامع وضعیت داده‌های آزمایشی (سناریوهای ۱ تا ۶)

پیش از بررسی تکی سناریوها، ذکر یک نکته سیستمی الزامی است. با بررسی لاگ‌های EXPLAIN ANALYZE در سناریوهای ۱ تا ۶ (به عنوان مثال سناریوی ۲ و ۴)، مشاهده می‌شود که زمان اجرا در حد کسری از میلی‌ثانیه است و خروجی اسکن‌ها به صورت actual time=... rows=0 ثبت شده است. این موضوع نشان می‌دهد که در لحظه اجرای این بنچمارک، یا حجم داده‌های موجود در جداول بسیار اندک بوده و یا رکوردهای وارد شده، شروط فیلتر (مانند title_type = 'movie') را ارضا نمی‌کرده‌اند. در چنین شرایطی، پایگاه داده با یک جدول تقریباً خالی روبرو است. افت‌های جزئی چند هزارم میلی‌ثانیه‌ای در سناریوهای ۳، ۵ و ۶ به این دلیل است که پیمایش ساختار درختی یک ایندکس روی یک جدول خالی، سربار (Overhead) محاسباتی اندکی نسبت به اسکن مستقیم و ترتیبی همان جدول خالی (Sequential Scan) دارد. با این وجود، برنامه‌ریز کوئری همچنان به درستی از ایندکس‌ها استفاده کرده است که در مقیاس داده‌های میلیونی، ارزش واقعی خود را نشان خواهند داد.

تحلیل سناریوی ۸: طولانی‌ترین فیلم‌ها به تفکیک ژانر (تحلیل بهبود چشمگیر)

این سناریو پیچیده‌ترین درخواست تحلیلی پروژه است. زمان اجرای این کوئری پیش از بهینه‌سازی ۸۵۲ میلی‌ثانیه و پس از ساخت ایندکس به ۶۹۵ میلی‌ثانیه کاهش یافت. خروجی اجرای موفق این سناریو در تصویر شکل ۱ قابل مشاهده است.

```

Incremental Sort (actual time=445.621..692.892 rows=133 loops=1)
-> Subquery Scan on ranked_movies (actual time=337.168..692.838 rows=133)
  -> WindowAgg (actual time=337.165..692.818 rows=133)
    -> Nested Loop Left Join (actual time=337.154..669.494 rows=434377)
      -> Gather Merge (actual time=337.124..503.121 rows=434377)
        Workers Planned: 2
      -> Sort (actual time=319.013..365.035 rows=144792 loops=3)
        Sort Method: external merge  Disk: 6920kB
      -> Nested Loop (actual time=79.883..138.086 rows=144792)
        -> Parallel Index Only Scan using idx_...
          (actual time=79.854..108.040 rows=90119 loops=3)
        Heap Fetches: 37757

```

genre	genre_rank	tconst	primary_title	runtime_minutes	start_year	average_rating
Action	1	tt0084052	The Hazards of Helen	1428	1914	
Action	2	tt0086206	Les Vampires	421	1915	
Action	3	tt0089204	The House of Hate	410	1918	
Action	4	tt0086917	Liberty	400	1916	
Action	5	tt0087932	The Fatal Ring	400	1917	
Adult	1	tt0073749	The Teenage Prostitution Racket	123	1975	
Adult	2	tt0078079	Bibi	120	1974	
Adult	3	tt0073806	Thundercrack!	120	1975	
Adult	4	tt0090652	Wild Man	117	1989	
Adult	5	tt0077590	Fabodjantan	115	1978	
Adventure	1	tt0083675	The Beloved Adventurer	450	1914	
Adventure	2	tt0085085	The Broken Coin	440	1915	
Adventure	3	tt0086206	Les Vampires	421	1915	
Adventure	4	tt0089701	Tih Minh	418	1918	
Adventure	5	tt0086917	Liberty	400	1916	
Animation	1	tt0069476	Voices do Medo	140	1972	
Animation	2	tt0077869	The Lord of the Rings	132	1978	
Animation	3	tt0088983	The Dagger of Kamul	132	1985	
Animation	4	tt0084881	Arcadia of My Youth	130	1982	
Animation	5	tt0088772	Galaxy Express 999	129	1979	
Biography	1	tt0076147	Hitler: A Film from Germany	442	1977	
Biography	2	tt0018192	Napoleon	330	1927	
Biography	3	tt0056937	Cleopatra	320	1963	
Biography	4	tt0062264	'Shtrikhi k portretu V. I. Lenina'	270	1967	
Biography	5	tt0095341	Hôtel Terminus	267	1988	
Comedy	1	tt0028785	Dick Tracy	290	1937	
Comedy	2	tt0036754	The Desert Hawk	285	1944	
Comedy	3	tt0066192	Out 1: Spectre	253	1972	
Comedy	4	tt0017159	The Adventures of the Three Reporters	250	1926	
Comedy	5	tt0025644	Perils of Pauline	231	1933	
Crime	1	tt0089905	Barrabas	489	1919	
Crime	2	tt0086206	Les Vampires	421	1915	
Crime	3	tt0027365	The Black Colt	323	1936	
Crime	4	tt0080991	The Carter Case	310	1919	
Crime	5	tt0029071	Jungle Menace	308	1937	
Documentary	1	tt0086930	The Journey	373	1987	

شکل ۱: خروجی اجرای موفق سناریوی ۸ (طولانی‌ترین فیلم‌ها به تفکیک ژانر)

۱. **Parallel Index Only Scan**: در حالت خط پایه، دیتابیس کل جدول `title_basics` را از روی دیسک اسکن می‌کرد که نیازمند خواندن حجم عظیمی از داده بود. در حالت بهینه‌شده، دیتابیس به `Parallel Index Only Scan` روی آورده است. از آنجا که ما فیلدهای `runtime_minutes` و `genres` را درون ایندکس گنجانده بودیم (`INCLUDE`)، پایگاه داده توانست تمام اطلاعات مورد نیاز خود را در ۱۰۸ میلی‌ثانیه فقط با خواندن ساختار فشرده‌اش.

۲. **Heap Fetches: 37757**: این عبارت نشان می‌دهد که با وجود استفاده از `Index Only Scan`، دیتابیس مجبور شده ۳۷۷۵۷ بار به جدول اصلی مراجعه کند. دلیل این امر این است که نقشه رؤیت‌پذیری (`Visibility Map`) کاملاً به‌روز نبوده و دیتابیس برای اطمینان از اینکه آیا رکورد یافته‌شده در ایندکس، در تراکش فعلی معتبر است یا خیر، به جدول اصلی (`Heap`) سر زده است. با یک بار اجرای دستی `VACUUM ANALYZE`، این عدد به صفر میل خواهد کرد و زمان کوئری باز هم کاهش می‌یابد.

۳. **Sort Method: external merge Disk: 6920kB**: این پیام بسیار مهم است. از آنجا که برای رتبه‌بندی رکوردها نیازمند مرتب‌سازی بودیم، دیتابیس تلاش کرده داده‌ها را در `RAM` مرتب کند. اما چون پارامتر `work_mem` ما کوچک بوده (حدود ۳۲ مگابایت)، حافظه پر شده و دیتابیس مجبور به ریزش داده‌ها به دیسک موقت (`Spill to Disk`) با حجم ۶۹۲۰ کیلوبایت شده است. این عملیات `I/O` دیسکی، عامل اصلی کندی این گره است. افزایش `work_mem` این گره را کاملاً به رم منتقل کرده و سرعت را چند برابر می‌کند.

۴. **Incremental Sort و WindowAgg**: تابع WindowAgg مسئول پردازش تابع پنجره‌ای (ROW_NUMBER() OVER (...)) بوده است که در مجموع ۶۹۲ میلی‌ثانیه به طول انجامیده و تنها ۱۳۳ خروجی (۵ فیلم برای هر ژانر) تولید کرده است. Incremental Sort نیز آخرین مرحله‌ای است که داده‌های نیمه‌مرتب‌شده را بهینه‌تر از یک Sort کامل، نهایی می‌کند [cite: 3].

تحلیل سناریوی ۲: ۱۰ فیلم برتر با بیشترین تعداد رأی (کالبدشکافی یک کوئری با خروجی تهی)

این سناریو به دنبال یافتن فیلم‌هایی است که حداقل ۱۰۰۰ رأی دارند. زمان خط پایه 0.032 ms و زمان بهینه‌شده 0.028 ms بوده است. بررسی لاگ این سناریو به ما نشان می‌دهد که چرا کوئری در کسری از میلی‌ثانیه اجرا شده است:

```
Limit (cost=8.46..8.47 rows=1 width=92) (actual time=0.016..0.017 rows=0 loops=1)
-> Sort (cost=8.46..8.47 rows=1) (actual time=0.015..0.016 rows=0 loops=1)
    Sort Key: tr.num_votes DESC, tb.tconst
    Sort Method: quicksort Memory: 25kB
-> Nested Loop (cost=0.43..8.45 rows=1) (actual time=0.002..0.002 rows=0)
    -> Seq Scan on imdb.title_ratings tr (actual time=0.002..0.002 rows=0)
        Filter: (tr.num_votes >= 1000)
```

۱. **Seq Scan on title_ratings**: پایگاه داده در پایین‌ترین سطح، جدولی که حاوی رتبه‌هاست را اسکن کرده است. قسمت **Filter: (tr.num_votes >= 1000)** تلاش کرده تا رکوردهای معتبر را جدا کند. اما مقدار **actual time=0.002..0.002** نشان می‌دهد که در ۰.۰۰۲ میلی‌ثانیه، هیچ سطری (rows=0) پیدا نشده است. این یعنی در داده‌های محدودی که به عنوان نمونه در دیتابیس بارگذاری کرده‌اید، هیچ فیلمی با بیش از ۱۰۰۰ رأی وجود نداشته است.

۲. **Nested Loop**: چون خروجی گره قبلی صفر سطر بوده، حلقه تودرتو (Nested Loop) که قرار بود این رکوردها را با جدول **title_basics** پیوند (Join) بزند، بلافاصله خاتمه می‌یابد و مقدار **actual time** آن نیز 0.002 ثبت شده است.

۳. **Sort**: گره مرتب‌سازی آماده بوده تا داده‌ها را بر اساس تعداد آرا مرتب کند. الگوریتم استفاده شده **quicksort** درون حافظه رم (Memory: 25kB) بوده است. از آنجا که هیچ رکوردی برای مرتب‌سازی وجود نداشته، این گره نیز در 0.016 میلی‌ثانیه بدون خروجی به پایان رسیده است.

۴. **چرا از ایندکس استفاده نشد؟** با وجود اینکه ما ایندکس **idx_title_ratings_top_votes** را ساخته بودیم، دیتابیس همچنان از اسکن ترتیبی (Seq Scan) استفاده کرده است. دلیل این امر هوشمندی Planner است؛ زمانی که پایگاه داده (بر اساس آمارهای جمع‌آوری شده توسط ANALYZE) می‌داند که حجم داده‌ها بسیار کم است یا خروجی وجود ندارد، از اسکن ترتیبی استفاده می‌کند زیرا بارگذاری ساختار درختی ایندکس سربار (Overhead) بیشتری نسبت به اسکن سریع یک جدول کوچک خواهد داشت.

تحلیل سناریوی ۷: محبوب‌ترین ترکیب دوتایی ژانرها (کالبدشکافی یک افتم عملکرد)

همیشه ایجاد ایندکس باعث بهبود نمی‌شود. این سناریو به دنبال یافتن متداول‌ترین ترکیب‌های دوتایی ژانرهاست. زمان خط پایه ۴۶۰ میلی‌ثانیه و پس از بهینه‌سازی ۵۱۰ میلی‌ثانیه ثبت شده است. تصویر خروجی این کوئری در شکل ۲ نمایش داده شده است.

```
Limit (actual time=490.239..490.244 rows=20 loops=1)
-> Sort (actual time=479.782..479.786 rows=20 loops=1)
    Sort Method: top-N heapsort Memory: 27kB
-> HashAggregate (actual time=479.714..479.742 rows=292 loops=1)
    -> Nested Loop (actual time=0.146..422.509 rows=267372 loops=1)
        -> Nested Loop (actual time=0.119..235.334 rows=350160 loops=1)
            -> Index Only Scan using idx_title_basics_movie_year
                Filter: (cardinality(tb.genres) >= 2)
                Heap Fetches: 37757
            -> Memoize (actual time=0.000..0.000 rows=2 loops=144316)
                -> Function Scan on unnest g1
```

۱. شکست در استراتژی اسکن: در حالت بهینه‌شده، برنامه‌ریز با دیدن ایندکس idx_title_basics_movie_year تصور کرده است که خواندن ایندکس ارزان‌تر از خواندن کل جدول است. بنابراین از Index Only Scan استفاده کرد که حدود ۱۴۲ میلی‌ثانیه زمان برد. اما چون فیلتر $\text{cardinality}(\text{genres}) \geq 2$ بود، رکوردهای بسیار زیادی همچنان انتخاب شدند و ایندکس مجبور شد به صورت پراکنده (Random Access) به حافظه رجوع کند (تولید ۳۷۷۵۷ Heap Fetches). در حالت خط پایه، دیتابیس از Parallel Seq Scan استفاده کرده بود که اگرچه کل جدول را می‌خواند، اما خواندن متوالی (Sequential Read Ahead) از روی دیسک، سرعت و توان عملیاتی بالاتری نسبت به پرش‌های تصادفی روی گره‌های ایندکس داشت.

۲. Memoize و unnest: از آنجا که استخراج عناصر یک آرایه با تابع unnest بسیار پرهزینه است، PostgreSQL از گره Memoize برای کش کردن خروجی این تابع استفاده کرده است. این گره ۱۴۴,۳۱۶ بار فراخوانی شده است.

۳. HashAggregate: پایگاه داده برای ترکیب و شمارش جفت ژانرها (GROUP BY genre_1, genre_2) از HashAggregate استفاده کرده است. این گره ۲۶۷,۳۷۲ سطر ورودی دریافت کرده و آن‌ها را در یک جدول هش درون رم (Memory Usage: 77kB) جمع کرده تا ۲۹۲ جفت یکتای ژانر را استخراج کند.

۴. top-N heapsort: در نهایت، چون کوثری شامل 20 LIMIT بود، گره مرتب‌سازی از الگوریتم بسیار بهینه top-N heapsort استفاده کرد. این الگوریتم به جای مرتب کردن کل ۲۹۲ رکورد، صرفاً یک درخت هیپ (Heap) با ظرفیت ۲۰ ایجاد کرده و داده‌ها را با کمترین مصرف حافظه (27kB) مرتب نموده است.

```

jasmin@Nothing:/media/jasmin/BA56C9E856C9A58B/DB/DB_Project_403102514_403110614_402106756
jasmin@Nothing:/media/jasmin/BA56C9E856C9A58B/DB/DB_Project_403102514_403110614_402106756
/media/j/B/DB/DB_Project_403102514_403110614_402106756 > awk '
/-- Scenario 7:/ {show=1}
/-- Scenario 8:/ {show=0}
show
' sql/queries.sql | \
docker compose exec -T postgres psql -U postgres -d indb
genre_1 | genre_2 | movie_count
-----
Drama   | Romance  | 6183
Crime   | Drama    | 5646
Comedy   | Drama    | 4711
Drama   | Western  | 4211
Comedy   | Romance  | 3912
Action   | Drama    | 3233
Adventure | Drama    | 2238
Action   | Adventure | 2843
Drama   | War       | 2015
Action   | Crime     | 1811
Comedy   | Crime     | 1661
Comedy   | Musical   | 1647
Drama   | Thriller  | 1460
Drama   | History   | 1376
Drama   | Mystery   | 1340
Adventure | Comedy   | 1150
Crime    | Mystery   | 1132
Action   | Comedy    | 1101
Biography | Drama     | 1019
Crime    | Thriller  | 992
(20 rows)

```

شکل ۲: خروجی اجرای موفق سناریوی ۷ (محبوب‌ترین ترکیب دوتایی ژانرها)

۵.۲. اسکرین شات از اجرای موفق

مطابق با نیازمندی‌های پروژه، در تصویر زیر خروجی ترمینال در زمان اجرای موفقیت‌آمیز کدهای پایتون (تولیدکننده و مصرف‌کننده پیام‌ها) و ارتباط صحیح آن‌ها با پایگاه داده PostgreSQL نشان داده شده است.

```

jasmin@Nothing:/media/jasmin/BA56C9E856C9A58B/DB/DB_Project_403102514_403110614_402106756
/media/j/B/DB/DB_Project_403102514_403110614_402106756 > python scripts/consumer.py \
--batch-size 100 \
--max-empty-polls 20 \
--no-purge-done
2026-08-08 22:35:41,725 | INFO | Processed 100 messages for indb.title_basics
2026-08-08 22:35:41,758 | INFO | Processed 100 messages for indb.title_basics
2026-08-08 22:35:41,788 | INFO | Processed 100 messages for indb.title_basics
2026-08-08 22:35:41,816 | INFO | Processed 100 messages for indb.title_basics
2026-08-08 22:35:41,845 | INFO | Processed 100 messages for indb.title_basics
2026-08-08 22:35:41,876 | INFO | Processed 100 messages for indb.title_basics
2026-08-08 22:35:41,908 | INFO | Processed 100 messages for indb.title_basics
2026-08-08 22:35:41,935 | INFO | Processed 100 messages for indb.title_basics
2026-08-08 22:35:41,965 | INFO | Processed 100 messages for indb.title_basics
2026-08-08 22:35:41,994 | INFO | Processed 100 messages for indb.title_basics
2026-08-08 22:35:42,026 | INFO | Processed 100 messages for indb.name_basics
2026-08-08 22:35:42,056 | INFO | Processed 100 messages for indb.name_basics
2026-08-08 22:35:42,089 | INFO | Processed 100 messages for indb.name_basics
2026-08-08 22:35:42,120 | INFO | Processed 100 messages for indb.name_basics
2026-08-08 22:35:42,152 | INFO | Processed 100 messages for indb.name_basics
2026-08-08 22:35:42,183 | INFO | Processed 100 messages for indb.name_basics
2026-08-08 22:35:42,215 | INFO | Processed 100 messages for indb.name_basics
2026-08-08 22:35:42,247 | INFO | Processed 100 messages for indb.name_basics
2026-08-08 22:35:42,276 | INFO | Processed 100 messages for indb.name_basics
2026-08-08 22:35:42,308 | INFO | Processed 100 messages for indb.name_basics
2026-08-08 22:36:01,425 | INFO | Queue remained empty for 20 polls; exiting
2026-08-08 22:36:01,426 | INFO | Consumer finished: 2000 successful messages, 0 NACKed messages, 0 invalid messages
/media/j/B/DB/DB_Project_403102514_403110614_402106756 >
  
```

شکل ۳: اجرای موفقیت‌آمیز کدهای Producer و Consumer

مراجع

PostgreSQL Global Development Group, “PostgreSQL 16 Documentation”. [۱]
<https://www.postgresql.org/docs/16/index.html>

PostgreSQL Global Development Group, “Resource Consumption (shared_buffers, work_mem, ...)”. [۲]
<https://www.postgresql.org/docs/16/runtime-config-resource.html>

PostgreSQL Global Development Group, “GIN Indexes”. [۳]
<https://www.postgresql.org/docs/16/gin.html>

PostgreSQL Global Development Group, “TOAST (The Oversized-Attribute Storage Technique)”. [۴]
<https://www.postgresql.org/docs/16/storage-toast.html>

PostgreSQL Global Development Group, “Explicit Locking (SELECT ... FOR UPDATE SKIP LOCKED)”. [۵]
<https://www.postgresql.org/docs/16/explicit-locking.html>